Assignment: Object Detection

Fine-tune pretrained object detection models, such as DETR, DINO, and RT-DETR, to detect piled garbage on the Rajshahi University (RU) campus using images captured with smartphones or digital cameras. Compare the models in terms of detection accuracy, computational efficiency, and robustness under varying campus conditions. Additionally, explore Grounding DINO for text-prompt-based garbage detection.

Assignment Tasks

1. Custom Dataset Collection

- Collect images of piled garbage from different locations across the Rajshahi University (RU) campus using a smartphone or digital camera.
- Include various types of garbage piles, such as household waste, plastic waste, paper waste, food waste, and mixed garbage.
- Capture images under different campus conditions, including:
    - Varying illumination and weather conditions.
    - Different backgrounds, such as roadsides, gardens, walkways, and areas near buildings.
    - Different viewing angles and camera positions.
    - Partial occlusion by vegetation, containers, or other objects.
    - Different garbage-pile sizes and distances from the camera.
    - Different garbage compositions and arrangements.
- Ensure that the dataset represents realistic garbage-disposal scenarios found across the RU campus.

2. Dataset Annotation and Preparation

- Annotate the garbage piles using bounding boxes and assign appropriate class labels.
- Begin with a single class, **piled garbage**. Optionally, define multiple classes to distinguish between different types of garbage.
- Divide the dataset into training, validation, and testing sets using an appropriate split.
- Ensure that the images represent diverse campus locations and environmental conditions.
- Where possible, separate images from particular locations or capture sessions across the dataset splits to reduce data leakage.
- Document the dataset size, class distribution, annotation procedure, and data-splitting strategy.

3. Data Augmentation

- Apply suitable data augmentation techniques to increase training-data diversity and improve detection robustness.
- Consider image flipping, scaling, cropping, translation, brightness and contrast adjustments, and other appropriate transformations.
- Ensure that bounding-box annotations remain consistent with the augmented images.
- Discuss how the selected augmentation techniques may improve detection under varying illumination, background complexity, and garbage-pile appearance.

4. Model Implementation and Fine-Tuning

- Select and implement suitable pretrained object detection models from the following architectures:
    - **DETR:** Detection Transformer.
    - **DINO:** DETR with Improved DeNoising Anchor Boxes.
    - **RT-DETR:** Real-Time Detection Transformer.
    - **Grounding DINO:** A language-guided object detector that supports text-prompt-based detection.
- Fine-tune the selected models using the custom RU campus garbage dataset, where supported by the chosen implementations and pretrained checkpoints.
- Document the model architectures, pretrained weights, training configurations, hyperparameters, computational resources, and number of training epochs.
- Investigate how different training configurations affect detection accuracy and computational efficiency.

5. Performance Evaluation

Evaluate each model on the same held-out test dataset using the following metrics:

- **Precision:** The proportion of predicted garbage detections that are correct.
- **Recall:** The proportion of ground-truth garbage piles successfully detected.
- **Mean Average Precision (mAP):** Evaluate detection accuracy using mAP@0.5 and, where feasible, mAP@0.5:0.95.
- **Inference speed:** Measure the number of images processed per second (FPS).
- **Detection latency:** Measure the time required to process an image and produce detections.

Use consistent evaluation conditions, including image resolution, hardware, confidence thresholds, and post-processing settings, wherever applicable.

6. Comparison with Original Pretrained Models

- Evaluate the original pretrained models on the same RU campus garbage-image test dataset before fine-tuning, where applicable.

- Compare the original and fine-tuned models using the evaluation metrics defined in Task 5.
- Analyze the improvements or changes in detection performance resulting from fine-tuning.
- Investigate whether fine-tuning improves garbage detection under challenging campus conditions.
- For Grounding DINO, evaluate zero-shot detection using text prompts such as *"a pile of garbage"*, *"a pile of plastic waste"*, and *"garbage dumped beside a campus road"*. Compare its zero-shot performance with its performance after adaptation, if supported by the selected implementation.

7. Comparative Analysis of Detection Architectures

- Compare DETR, DINO, and RT-DETR in terms of detection accuracy, inference speed, computational requirements, and robustness in piled garbage detection.
- Investigate the effects of garbage-pile size, composition, illumination, occlusion, viewing angle, and campus background on detection performance.
- Evaluate Grounding DINO's ability to detect different garbage piles using natural-language prompts and discuss its advantages and limitations compared with the other detectors.
- Identify challenging cases involving false positives, false negatives, missed detections, and inaccurate bounding boxes.
- Discuss the trade-offs between detection accuracy, computational cost, and inference efficiency.

8. Optional Extension: Text-Prompt-Based Garbage Detection

- Develop an interactive interface that accepts an RU campus image and a natural-language prompt describing the garbage to be detected.
- Use Grounding DINO to identify garbage piles matching the prompt.
- Display detected garbage piles with bounding boxes and confidence scores.
- Test different prompts describing the same or different garbage types and analyze how prompt wording affects detection performance.

Report Requirements

Prepare a technical report containing the following sections:

1. Introduction and Objectives
2. Related Work
3. Dataset Collection, Preparation, and Annotation
4. Data Augmentation Strategies
5. Model Architectures and Implementation
6. Experimental Setup and Training Configuration

7. Quantitative Results
    - Precision
    - Recall
    - mAP@0.5 and mAP@0.5:0.95, where applicable
    - Inference speed (FPS)
    - Detection latency
8. Comparison Between Original Pretrained and Fine-Tuned Models
9. Comparative Analysis of DETR, DINO, RT-DETR, and Grounding DINO
10. Qualitative Detection Results
11. Performance Analysis, Strengths, and Limitations
12. Discussion
13. Conclusion
14. References

Include representative detection visualizations, quantitative comparison tables, training curves where available, and examples of successful and unsuccessful detections.

Deliverables

1. **Source code:** Complete, documented, and reproducible implementation.
2. **Custom dataset:** Images collected on the RU campus, bounding-box annotations, and dataset-splitting information.
3. **Trained models:** Fine-tuned model weights and training configurations.
4. **Evaluation results:** Quantitative metrics and qualitative detection examples.
5. **Comparative analysis:** A systematic comparison of the selected architectures.
6. **Technical report:** A structured report documenting the methodology, experiments, results, and conclusions.

Important Implementation Notes

- DETR, DINO, and RT-DETR support supervised object detection using bounding-box annotations, provided the selected implementations and pretrained checkpoints support the required training procedure.
- Grounding DINO supports open-set, text-prompt-based object detection. Its original pretrained model can detect objects described by text without conventional class-specific fine-tuning. Fine-tuning requires an implementation and training procedure that supports the intended adaptation.
- For a fair comparison, use the same held-out test images and consistent evaluation criteria. Note that Grounding DINO uses text prompts, whereas the other detectors generally predict from predefined class sets.
- Record the hardware and inference settings used for each model.
- Obtain appropriate permission before photographing campus locations, and avoid unnecessarily capturing identifiable individuals.